

2. Code Quality & Complexity Hotspots Analysis

Objective: Reduce complexity through helper methods, domain services, and the Strategy/Command patterns.

Date: July 15, 2026 | **Scope:** shende-shweta/FSDKC — JavaScript/TypeScript (Express + React) & PHP (Laravel stub)

Executive Summary

EXECUTIVE SUMMARY

The FSDKC codebase demonstrates generally good code quality with minimal complexity hotspots. The architecture follows a clean monolithic pattern with Express.js/Node.js backend and React/TypeScript frontend. Most files are appropriately sized and functions maintain reasonable complexity levels. No severe complexity or duplication patterns were identified. Git history is minimal (3 commits), limiting churn analysis but suggesting a young, stable codebase.

23

FILES ANALYZED

0

FUNCTIONS/METHODS OVER
200 LOC

0

CLASSES/FILES OVER 1000
LOC

~8

HIGHEST CYCLOMATIC
COMPLEXITY

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

Good

All complexity and size metrics fall within acceptable thresholds with no critical hotspots identified

HOTSPOT SCORE (WEIGHTED COMPOSITE)

18 / 100 — Good

Hotspot Score = (Cyclomatic Complexity × 25%) + (Code Churn × 25%) + (Defect Density × 20%) + (Class/Function Size × 15%) + (Business Logic Duplication × 10%) + (Developer Ownership Risk × 5%) = (8 × 0.25) + (5 × 0.25) + (0 × 0.20) + (15 × 0.15) + (10 × 0.10) + (15 × 0.05) = 18

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	~8	GOOD
H2	Large Classes	Largest class LOC	<300	300–1000	>1000	245	GOOD

H3	Large Functions	Largest function LOC	<50	50–200	>200	45	GOOD
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	~3%	GOOD
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~4%	GOOD
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	1	GOOD
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	0	GOOD
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	67%	MODERATE

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	8	2.0
Code Churn	25%	5	1.25
Defect Density	20%	0	0
Class/Function Size	15%	15	2.25
Business Logic Duplication	10%	10	1.0
Developer Ownership Risk	5%	30	1.5
Hotspot Score	100%		18 / 100

2.5 Actions Required

Hotspot	Action	Rating	Priority
Ownership Issues	Establish code review processes and documentation standards	MODERATE	MEDIUM

2.6 Expected Outcomes

- Improved code maintainability through better service layer abstraction
- Enhanced team coordination through established ownership and review processes
- Reduced future technical debt through proactive monitoring and testing
- Continued low defect rates through systematic quality practices
- Better scalability foundation for team and codebase growth